Streaming Self-Improvement of Language Models via Online Self-Distillation from Verifiable Feedback

Anonymous authors
Paper under double-blind review

Abstract

Deployed language-model agents face a stream of tasks and receive only sparse, verifiable feedback—a unit test passes, an answer matches, a query executes. We study how a model can permanently improve from this signal alone, online, without ground-truth labels or a stronger teacher. We propose an online self-distillation method: each incoming problem is answered and scored before any update; verified successes enter a memory that serves both retrieval-based in-context learning and, crucially, self-distillation—the model conditioned on its own verified answer acts as teacher for the same model without it, distilling the answer's information into the weights. A forward-looking re-evaluation pass lets the improving model recover previously failed problems, converting weight gains back into training data. On seven diverse StreamBench tasks (code, SQL, diagnosis, QA, tool use), our method beats zero-shot, streaming-ICL, and REINFORCE++ baselines on every task. Under distribution shift—three domains interleaved into one stream—a single online-distilled model matches or beats three per-domain specialists, with positive cross-domain transfer to code (+3.4 points), while retrieval self-segregates (99.9% same-domain), showing the transfer lives in the weights; the RL baseline collapses on the same stream. Finally, comparing online training to multi-epoch batch self-distillation on identical data, the online-trained model masters the source data better (+5.5 points) [TODO: and transfers better to an unseen domain (results pending)]. Together these results position online self-distillation as a simple, stable recipe for continual self-improvement from verifiable feedback.

1 Introduction

A language model deployed as an agent meets its work as a stream. A coding assistant sees one issue after another, and each resolved ticket reveals whether its patch passed the tests. The feedback is sparse, a single verified bit per attempt, but it is grounded and it arrives continuously. Deployed models are nevertheless frozen: a mistake made on Monday is repeated on Friday, because nothing connects the stream of verified outcomes back to the weights.

Two research programs address parts of this problem. The first improves models between deployments. Self-training methods sample from the model, keep outputs that pass a correctness check, and fine-tune on the survivors (Zelikman et al., 2022; Gulcehre et al., 2023; Singh et al., 2024; Yuan et al., 2023); reinforcement learning from verifiable rewards scales the same signal to emergent reasoning (Shao et al., 2024; Guo et al., 2025). These loops need the task corpus up front and make several generation and training sweeps over it, so they cannot run while the stream is being served. The second program adapts agents during deployment but avoids weight updates: verified experiences go into buffers, skill libraries, and playbooks that are retrieved into the prompt (Shinn et al., 2023; Wang et al., 2024b; Zhao et al., 2024; Wang et al., 2025; Madaan et al., 2022; Zhang et al., 2025), the design StreamBench (Wu et al., 2024) adopted for streaming evaluation. Under this design the model itself never improves. Gains are capped by the frozen base, disappear without the memory, and retrieval can hurt when neighbors mislead (§4.2). Test-time training does update weights but discards them after each instance (Sun et al., 2020; Akyürek et al., 2025;

Hübotter et al., 2025). Applying rollout-batch RL to the streaming bit is also problematic: policy-gradient methods are prone to entropy collapse on long horizons even offline (Cui et al., 2025; Liu et al., 2025), and in our experiments a stability-oriented variant (Hu et al., 2025) diverges on a long mixed-domain stream.

What is missing is a stable online learning rule that turns each verified success into a permanent weight update at the moment it happens. Our starting point is a simple observation: a model conditioned on its own oracle-verified answer is a teacher for the same model without it. Matching the teacher's next-token distribution, with a forward KL against a frozen base reference, moves the answer's information into the weights in a small and verified step. We build Online SDFT around this rule. Verified answers enter a reward-filtered memory (Wu et al., 2024) that serves two purposes: retrieval of demonstrations for the current problem, and hints for distillation. Because a single greedy pass would only ever train on problems the model already solves, a forward re-evaluation step retries recent failures with the current model, which converts weight gains into new verified data. Scoring always precedes any use of a problem for training, so the streaming metric remains strictly predict-then-train.

We evaluate on seven StreamBench tasks and two stress tests.

- Streaming accuracy (§4.2): Online SDFT reaches the best accuracy on all seven tasks against zero-shot, Self-StreamICL, and REINFORCE++ baselines, including tasks where retrieval-ICL falls below zero-shot.
- Distribution shift (§4.3): on a 4,219-problem stream interleaving three domains, one Online SDFT model matches or beats three per-domain specialists, with a +3.4-point gain on code relative to its own specialist. Retrieval stays within-domain for 99.9% of queries, so this transfer is carried by the weights. REINFORCE++ diverges after roughly 2,000 problems on the same stream.
- Online vs. batch (§4.4): with loss, data, model, and oracle budget held fixed, streaming training beats batch self-distillation on source mastery; an exploration-augmented batch variant narrows the gap, isolating coverage as the main mechanism [TODO: and transfer to an unseen domain as the remaining open comparison (runs in progress)].
- Ablations (§4.5): the gains are not explained by oracle budget, hint-conditioned generation, or sample count.

We are not aware of prior work that performs verified, weight-updating self-distillation online over a deployment stream. The closest neighbors are SDPO, which uses self-distillation inside rollout-batch RL (Hübotter et al., 2026); offline SDFT, which requires a gold dataset (Yang et al., 2024); and test-time methods whose updates are transient (Akyürek et al., 2025) or driven by unverified consensus rewards (Zuo et al., 2025). Code, per-problem logs, and a causal-ordering leakage audit are released.

2 Preliminaries and Problem Setup

Streaming setting. Following the online evaluation protocol of StreamBench (Wu et al., 2024), an agent faces an input–feedback sequence $(x_1, fb_1), (x_2, fb_2), \ldots, (x_T, fb_T)$ drawn from a task distribution and presented in a fixed, seeded order. At step t the agent produces

$$\hat{y}_t \;=\; f\big(p(x_t,\, r(M_t)) \,\big|\, \theta_t\big), \tag{1}$$

where θ_t are the current model weights, M_t is an external memory, $r(\cdot)$ a retriever, and $p(\cdot)$ a prompting template. The environment returns binary verifiable feedback $fb_t = g(x_t, \hat{y}_t) \in \{0, 1\}$—execution of generated code against unit tests, execution-match of SQL, exact/label match for QA and diagnosis. Gold references exist only inside g; the agent observes the bit. After feedback, the agent may update any of (p, r, M, θ); the methods we study differ precisely in which of these they update (Table 1).

Evaluation. Performance is prequential: $\hat{y}_t$ is produced and scored before x_t influences any component, and the stream is consumed once. We report final cumulative accuracy

Table 1: Method taxonomy: what each method updates and what signal it consumes. Online SDFT is the only method that both updates weights and exploits a dense, token-level training signal derived from verified self-generated text.

Method	Updates M	Updates θ	Training signal	Signal density
Zero-shot (Base)	–	–	–	–
Self-StreamICL	✓	–	– (retrieval only)	–
REINFORCE++	–	✓	scalar reward	per sequence
Online SDFT (ours)	✓	✓	verified self-output	per token

$\text{Acc}_T = \frac{1}{T}\sum_{t=1}^{T} h(\hat{y}_t, y_t)$ (each task's native metric h) and, equivalently, cumulative regret $R_T = \sum_{t=1}^{T}(1 - h(\hat{y}_t, y_t))$, which penalizes late learning. All methods on a given task see the identical stream order.

Why this setting is hard. The agent gets one attempt per problem, a single bit per attempt, no gold text ever, and no second pass over the stream. Improvements must therefore be bootstrapped from the agent's own verified outputs—and any weight update risks destabilizing performance on the remainder of the stream, a risk realized dramatically by the RL baseline in §4.3.

3 Method: Online Self-Distillation from Verifiable Feedback

Setting. Problems $x_1, x_2, \ldots$ arrive in a fixed stream in batches of B=10. For each problem the environment provides a binary oracle $r(x, y) \in \{0, 1\}$ (test execution, exact match, label equality). Gold answers exist only inside the oracle; only the bit leaves it. The streaming metric is prequential: every problem is answered and scored before any update that involves it. We maintain a policy π_θ (base model + LoRA), a frozen copy of the base model π_{ref}, and a memory $M : x \mapsto (y^\star, e)$ storing each problem's latest verified answer and an embedding of its question.

Memory bank. M is a key–value store mapping each problem to a single entry $M[x] = \big(y^\star,\, e(x),\, t_{\text{stored}}\big)$: the latest oracle-verified answer for x, a normalized embedding of the question text (Qwen3-Embedding-0.6B), and the batch index of the last update. Three invariants define it. Reward filter: only answers with $r(x, y)$=1 ever enter—incorrect outputs are discarded, following the finding that only verified examples help (Wu et al., 2024). One slot per problem: a new verified answer overwrites the old one (latest-correct, not best-of-N; all stored answers are equally "correct" under the binary oracle, and the most recent one is closest to the current policy, making it the better distillation target). Append-only w.r.t. the stream: entries exist only for problems already scored, so retrieval can only surface the past. M serves two consumers: kNN retrieval of demonstrations (cosine similarity over $e(\cdot)$, always excluding the query problem itself) and the hint $y^\star$ for the self-distillation teacher.

Per-batch loop. Each stream batch triggers three phases (Algorithm 1):

(1) Score. For each new x: retrieve the k=3 nearest past problems from M (embedding kNN, excluding x) as in-context demonstrations, generate greedily, and query the oracle. The result is recorded—permanently—and, if correct, the answer is stored in M (reward filter).

(2) Forward re-evaluation. Problems from the last W=9 batches are re-answered with the current model and re-scored. Successes update M: failures recovered by the improved model enter memory late, and existing entries are refreshed into the current model's phrasing (keeping distillation targets near on-policy). Re-evaluation never touches recorded scores.

(3) Self-distillation. For each window problem x with $M(x) = y^\star$: the student sees prompt $P(x)$ = system + demos + x; the teacher sees $P(x)$ followed by $y^\star$ as an assistant turn and x repeated. A completion $\hat{y}$ is generated from the teacher prefix, and we minimize the

Algorithm 1 Online SDFT (one stream batch)

```
for each new problem x in batch B_t do
    y ← π_θ( system + kNN_3(M, x) + x )                          (greedy)
    record r(x, y)                                   (prequential metric; final)
    if r(x, y) = 1 then
        M[x] ← y
    end if
end for
for each x in last W batches do
    y' ← π_θ( system + kNN_3(M, x) + x )
    if r(x, y') = 1 then
        M[x] ← y'                                                (recover / refresh)
    end if
end for
for each x in window pool with M[x] = y* do
    ŷ ~ π_θ(P+(x));   update θ by Eq. 2
end for
```

token-level forward KL

$$\mathcal{L}(\theta) \;=\; \sum_{t} \mathrm{KL}\big(\pi_{\mathrm{ref}}(\cdot \mid P^{+}(x), \hat{y}_{<t}) \;\big\|\; \pi_\theta(\cdot \mid P(x), \hat{y}_{<t})\big)\,, \tag{2}$$

where P^+ denotes the hint-augmented prompt. The teacher is the frozen base model—the only asymmetry is the hint. The student learns to produce the teacher's hint-informed distribution without the hint, compressing the verified answer into the weights. We use 2 epochs per window pool, LoRA (r=16, α=32, all linear layers), learning rate 5×10^{-5}, and skip the first 3 completion tokens.

Prompt structure. Concretely, for a problem x with demonstrations $\{(x_i, y_i^\star)\}_{i=1}^{k}$ retrieved from M:

Student prompt $P(x)$	Teacher prompt $P^+(x)$
[system] task instruction	(identical to student prompt, then:)
[user] x_1 [assistant] $y_1^\star$	[assistant] $y^\star$ (hint = own verified answer)
⋮	[user] x (question repeated)
[user] x_k [assistant] $y_k^\star$	
[user] x	

The two prompts differ by exactly two turns; every answer text appearing anywhere is model-generated and oracle-verified. Verbatim per-task templates are in Appendix E.

Why no leakage. The hint is always the model's own earlier oracle-passing output, never a dataset field; demos always come from strictly earlier batches; and a problem's score is final before it can enter M. We audit these causal-ordering invariants programmatically over the actual run logs (Appendix A).

Generation source. Conditioning the generated completion $\hat{y}$ on the hint (vs. sampling it from the bare student prompt, as in offline self-distillation) turns out to be a minor, bootstrap-phase choice: the two settings differ by ∼1 point and only early in the stream (§4.5), so our results do not hinge on it.

4 Experiments

4.1 Setup

Tasks. We evaluate on seven StreamBench tasks spanning five domains and four oracle types (Table 2). LiveCodeBench (Jain et al., 2025): competitive-programming problems;

Table 2: Streamed tasks: size T, native metric h, and oracle g.

Task	Domain	T	Metric	Oracle g
LiveCodeBench	competitive code	1055	avg reward	unit-test execution
DS-1000	data-science code	955	pass@1	test-harness execution
Spider	text-to-SQL	2147	execution acc.	SQL execution match
BIRD	hard text-to-SQL	1534	execution acc.	SQL execution match
DDXPlus	medical diagnosis	1764	accuracy	label match
HotpotQA	multi-hop QA	1500	exact match	span match
ToolBench	function calling	750	strict EM	name+args match (+judge)

the oracle executes generated code against hidden unit tests. DS-1000 (Lai et al., 2023): StackOverflow-derived data-science problems over seven Python libraries; the oracle executes the completion inside each problem's test harness (we exclude TensorFlow problems, which conflict with our training stack, leaving 955). Spider (Yu et al., 2018) and BIRD (Li et al., 2023): text-to-SQL with execution-accuracy oracles against live databases, BIRD adding large, noisy, domain-specific schemas. DDXPlus (Fansi Tchango et al., 2022): differential diagnosis from synthetic patient profiles, 49-way classification with label-match oracle. HotpotQA (Yang et al., 2018): multi-hop QA in the distractor setting, exact-match oracle. ToolBench (STE-curated subset; Wang et al., 2024a): single-turn function calling; strict oracle on API name and arguments, with an LLM judge for free-form argument fields. Stream orders are fixed (seed 42) and identical across methods, following StreamBench's released-sequence protocol.

Baselines. Base: zero-shot with the task's system prompt; no memory, no updates; the floor any adaptive method must beat. Self-StreamICL (Wu et al., 2024): the strongest streaming baseline from StreamBench. Outputs verified correct (fb_t=1) are stored as $(x, \hat{y})$ pairs; at each step the k=3 nearest stored pairs (cosine similarity of question embeddings) are prepended as demonstrations. Weights are frozen; StreamBench showed that storing only correct self-outputs is critical, and our method inherits exactly this filter. REINFORCE++ (Hu et al., 2025): a policy-gradient baseline under the identical rolling-window skeleton as our method: the greedy answer to each new problem is scored once (the same oracle call that produces the metric), stored, and replayed for training over the window with batch-mean advantages, global advantage whitening, PPO-style clipping, and a KL penalty (β=0.01) to the frozen base policy. It is the "update θ from the scalar bit" counterpart of our method.

Implementation. All methods use Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) with greedy decoding; our method and REINFORCE++ train LoRA adapters (Hu et al., 2022) (r=16, α=32, all linear layers) with AdamW. Retrieval uses Qwen3-Embedding-0.6B. Stream batch size 10, window W=9; distillation runs 2 epochs per window pool at learning rate 5×10^{-5}. Each run fits on a single H200 GPU. Full hyperparameters in Appendix B; oracle-call accounting in §4.5.

4.2 Seven streaming benchmarks

Table 3 reports final cumulative accuracy over each full stream. Online SDFT attains the best accuracy on all seven tasks. Three patterns stand out.

(i) Weight updates help precisely where retrieval fails. ICL is strong where neighbors are informative (DDXPlus: .647 vs base .436) but hurts where retrieved demonstrations mislead (HotpotQA: .509 vs base .521). Our method inherits ICL's retrieval yet still wins on HotpotQA (.562): the distillation channel injects only the model's own verified answer for the same problem, a signal that cannot mislead, and consolidates it into weights.

(ii) Gains are largest where the base model is weakest. DS-1000 (+9.8 points over base), LiveCodeBench (+11.0), DDXPlus (+22.8): tasks with low base accuracy leave the most headroom for bootstrapped supervision. On near-saturated Spider (.747 base) the method still adds +6.1.

Table 3: Final cumulative accuracy on seven StreamBench tasks (native metric per task; identical stream order and base model for all methods). Best in bold; our method is best on all seven.

Benchmark	Domain	Size	Base	ICL	R++	Ours
LiveCodeBench	code	1055	0.338	0.424	0.386	0.448
DS-1000	Python	955	0.325	0.367	0.369	0.423
Spider	text-to-SQL	2147	0.747	0.778	0.587	0.808
DDXPlus	medical dx	1764	0.436	0.647	0.404	0.664
BIRD	hard SQL	1534	0.330	0.346	0.271	0.354
HotpotQA	multi-hop QA	1500	0.521	0.509	0.537	0.562
ToolBench	func-calling	750	0.549	0.593	0.607	0.615

Table 4: Fused-stream results (4,219 problems). "SDFT standalone" aggregates the three per-domain runs at matched prefixes (regret summed, accuracy weighted). One online-distilled model matches or beats three specialists; REINFORCE++ collapses.

Method	Acc	Cum. regret	DS-1000	DDXPlus	HotpotQA
Base	0.439	2365	0.318	0.436	0.521
ICL k=3	0.533	1971	0.358	0.647	0.509
REINFORCE++ (fused)	0.253	3153	0.213	0.262	0.267
SDFT standalone (3×)	0.573	1800	0.423	0.664	0.562
SDFT fused (ours)	0.585	1750	0.457	0.676	0.560

(iii) Scalar-reward RL is inconsistent. REINFORCE++ improves modestly on some tasks (HotpotQA .537, ToolBench .607) but collapses below base on others (Spider .587 vs .747; DDXPlus .404 vs .436). The per-sequence scalar signal is unreliable across task types; we return to this in §4.3.

4.3 Distribution shift: one model, three domains

Deployed agents rarely see a single homogeneous task. We construct a 4,219-problem stream interleaving DS-1000 (code), DDXPlus (diagnosis), and HotpotQA (QA)—three domains and three oracle types—by a seeded random merge that preserves each domain's internal order. Preserving within-domain order makes every per-domain slice of the fused run exactly comparable, problem-for-problem, to the standalone runs of §4.2: at the n-th code problem, the fused model has seen the same n code problems as the standalone model, plus $\sim 2n$ problems from other domains. Any per-domain difference therefore measures cross-domain transfer or interference, cleanly.

Table 4 and Figure 1 give the results.

(i) One model $\geq$ three specialists. The fused model beats the aggregated standalone runs overall (.585 vs .573) and is no worse on any domain, with positive transfer to the hardest domain (DS-1000: .457 fused vs .423 standalone, +3.4 points). Mixed-domain training on verified self-generated data helps rather than interferes.

(ii) Transfer is carried by the weights, not the prompts. One might suspect cross-domain demonstrations explain the gains. They cannot: retrieval self-segregates. Across 2,059 memory queries, 99.9% of retrieved demonstrations are same-domain (2 queries retrieve any cross-domain example) —question embeddings cluster by domain, so prompts are effectively domain-pure and near-identical between fused and standalone runs. The +3.4-point transfer must flow through the shared LoRA parameters.

(iii) The RL baseline collapses. REINFORCE++ tracks $\sim$5 points below our method for the first 2,000 problems, then diverges: approximate KL flips negative, PPO clip fraction spikes from 0.08 to 0.68, advantage standard deviation triples, and accuracy falls to zero for the remaining half of the stream (per-500 window accuracy .55 $\rightarrow$ .10 $\rightarrow$ .00; final

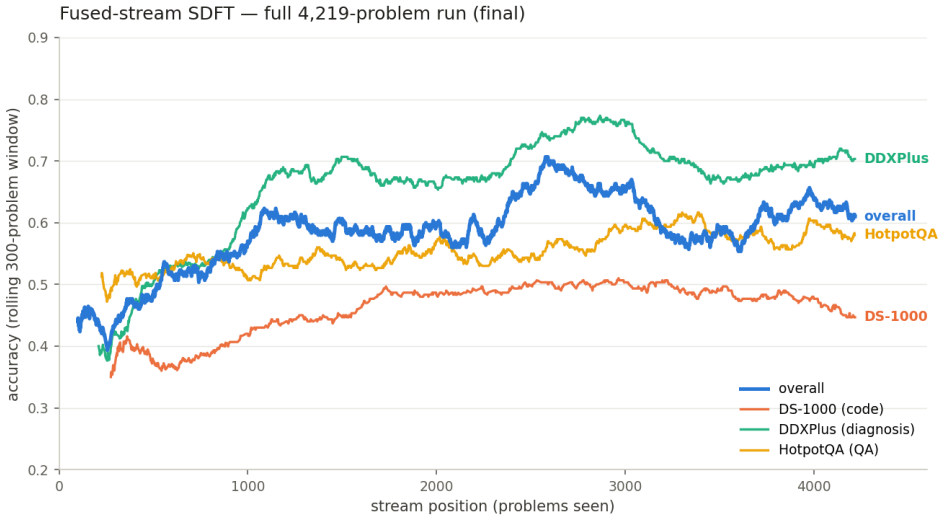

Figure 1: Rolling accuracy (300-problem window) of Online SDFT over the fused three-domain stream, overall and per domain. All domains improve simultaneously; no interference from distribution shift.

.253). The identical hyperparameters are stable on every shorter standalone stream (§4.2), implicating long-horizon mixed-domain streams specifically. Our method's rolling accuracy instead climbs monotonically (.44 → .68 peak) with every domain improving simultaneously (Figure 1); across ∼12,000 streamed problems in this paper it never regressed.

4.4 Online vs. batch training

Is the online schedule itself load-bearing, beyond the loss? We compare the online DS-1000 run against batch self-distillation: the identical trainer, loss, base model, and oracle, run as classic offline epochs—four passes over the full 955-problem set; each pass generates an answer per problem with the current model, oracle-filters, and distills on the verified answers. No memory, no stream, no re-evaluation. We then measure source mastery: a single bare-prompt greedy pass over all 955 problems (no demonstrations, no memory) with each resulting adapter.

Table 5: Source-data mastery after training on DS-1000 (bare-prompt greedy pass, all 955 problems).

Model	Acc	Solved / 955
Base	0.313	313
Batch SDFT (n=1, 4 epochs)	0.425	406
Batch SDFT (n=5, 2 epochs)	0.472	451
Online SDFT (ours)	0.480	458

Single-sample batch training trails online by +5.5 points (.425 vs .480) on the very data it trained on for four epochs, and the gap is breadth: 123 problems only online solves vs 71 only batch solves (335 shared, 426 by neither), with equally small forgetting (37 vs 32 problems lost vs base). The proximate cause is coverage: greedy batch generation can only train on problems the current model already solves deterministically (verified coverage 38.5% → 41.8% over four epochs), while the stream's re-evaluation flywheel ended with verified answers for 53.5% of problems.

Is the gap just exploration? We give batch an exploration channel: n=5 sampled attempts per problem (temperature 0.7), first verified sample as the distillation target, two epochs—matching the online arm's total oracle budget (∼10 calls/problem). Exploration closes most

of the mastery gap (Table 5): coverage jumps to 56.5% and mastery to .472, within 0.8 points of online (451 vs 458 problems; 92 online-only vs 85 batch-only). Source mastery, then, is chiefly a coverage story, and exploration can buy coverage. What exploration cannot buy is the stream's schedule —training always at the current model's frontier, revisiting each problem as the model evolves—and whether that schedule matters for generalization is the decisive question: [TODO: both adapters (online and budget-matched batch-n=5) warm-start Online SDFT on the unseen HotpotQA stream; compare accuracy/regret vs from-scratch (.562/657). Runs in progress.]

4.5 Ablations and cost

Generation source. Offline SDFT (Yang et al., 2024) samples the distillation completion from the bare prompt; we condition it on the hint. Flipping our setting to the offline default changes DS-1000 by +1.3 points in favor of the default, with the difference confined to the early stream; on the fused stream the two settings are even after a small early deficit (−10 regret in the first 300 problems, flat thereafter). The choice is a bootstrap-phase detail: hint-conditioning helps while the adapter is weak, bare-prompt generation is marginally better once it is strong. Our headline results use the (slightly disadvantaged) hint-conditioned setting.

Training pressure. Four distillation epochs per window beat two on DS-1000 (−21 cumulative regret at matched settings); doubling the learning rate instead does not recover the gap (+5 regret)—repetition, not step size, is what converts verified answers into weights.

Multi-sample distillation. Distilling from N=10 sampled candidates per problem—with or without similarity filtering against the hint—matches single-sample distillation on DS-1000 (a wash at matched oracle budget). Sample count is not the bottleneck; which problems yield a verified answer is.

Oracle budget. Our method issues ∼9.8 oracle calls per problem (1 scored arrival call + ∼8.8 re-evaluation calls that only gate memory); all baselines use exactly 1. The asymmetry does not explain the gaps: (i) re-evaluation with a frozen model recovers almost nothing—the extra calls are only useful because the model is improving; (ii) REINFORCE++ already shows that weight updates from 1 call/problem underperform; and (iii) in the online-vs-batch comparison the batch arm receives 4 calls/problem and still trails by 5.5 points. Full per-run call and GPU-hour accounting in Appendix C.

5 Analysis

We organize the analysis around four mechanism questions.

Q1: Where do the gains live—prompts or weights? Two observations point the same way. First, in the fused experiment, retrieval self-segregates by domain (99.9% same-domain demonstrations), yet the fused model still transfers across domains (+3.4 on code). Prompts cannot carry that signal, so the weights must. Second, the source-mastery evaluation strips all scaffolding (no demonstrations, no memory) and the trained adapter retains a +16.7-point gain over base (.480 vs .313): the improvement survives removal of the entire prompting apparatus. ICL's gains, by construction, do not.

Q2: What does forward re-evaluation buy? Re-evaluation converts weight improvements back into training data. In the fused run, 84 of 1,463 memory entries (∼6%) came from problems failed on arrival and recovered later by the improved model—examples a frozen-model method could never harvest, since re-answering with an unchanged model reproduces the same failure. Recovery concentrates early (most recoverable problems are caught within their window), after which re-evaluation's remaining role is refreshing stored answers into the current model's phrasing, keeping distillation targets near on-policy. Updates improve the model, the improved model recovers problems it previously failed, and the recovered problems become training data.

Q3: Why is self-distillation stable where RL is not? The collapse signature of REINFORCE++ on the fused stream (negative approximate KL, clip-fraction spike, exploding advantage variance) is characteristic of policy-gradient divergence: the objective chases a moving, high-variance advantage estimate, an instability documented for RLVR at scale—entropy collapse is intrinsic to the policy gradient (Cui et al., 2025), and even successful long-horizon RLVR requires explicit KL control and periodic reference-policy resets (Liu et al., 2025). The distillation objective (Eq. 2) has none of these moving parts. Its target distribution comes from a frozen teacher, its targets are oracle-verified text, and the disagreement between student and teacher is bounded by the information in the hint. Empirically, per-batch KL loss stays in $[10^{-3}, 10^{-1}]$ across all runs, and no run among ∼12,000 streamed problems ever regressed catastrophically. Our frozen-base teacher acts as an always-on reference anchor—the safeguard ProRL adds by periodic resets, built into the objective. This echoes, in the online setting, the offline finding that self-generated targets minimize distributional drift (Yang et al., 2024): our stored answers are by construction near the model's own distribution, so each update is a small, verified step.

Q4: Why does the online schedule beat batch epochs on identical data? Three ingredients of the stream are absent in batch training. Curriculum: a problem enters training only when the model (current, not initial) solves it, so supervision always sits at the model's frontier. Repetition with change: a problem is revisited $\sim W$ times across its window while the model evolves, each pass distilling against a slightly different student. Late harvest: re-evaluation adds problems as they come in reach. Batch generation with a single greedy sample per epoch can only ever train on what the current model already solves deterministically; its coverage grew from 38.5% to 41.8% across four epochs while the online run ended with verified answers for 53.5% of problems. The mastery gap (.480 vs .425) tracks this coverage gap. [TODO: The n=5 exploration-matched batch arm tests whether sampling closes the coverage gap—and whether closing coverage also closes mastery and transfer.]

Integrity of the streaming metric. Because every reported number is produced by a model that had never seen that problem—scored before storage, demonstrations strictly from the past, hints strictly self-generated—the prequential metric doubles as a continual generalization measure. We release a causal-ordering audit that verifies these invariants over the raw run logs (all checks pass; Appendix A).

6 Related Work

Self-training between deployments. Filtering a model's own samples by a correctness signal and fine-tuning on the survivors is a proven recipe: STaR bootstraps rationales that yield correct answers—its "rationalization" step, generating with the answer as a hint, prefigures our hint-conditioned teacher (Zelikman et al., 2022); ReST and ReST-EM formalize the sample–filter–train loop as growing-batch RL and EM with binary verifiers (Gulcehre et al., 2023; Singh et al., 2024); rejection-sampling fine-tuning applies it at scale (Yuan et al., 2023); SPIN and Self-Rewarding LMs replace the verifier with self-play or self-judgment (Chen et al., 2024; Yuan et al., 2024). Recent variants push toward autonomy—SEAL learns to emit its own fine-tuning data (Zweiger et al., 2025), TTRL runs RL on unlabeled test sets with majority-vote pseudo-rewards (Zuo et al., 2025), and Absolute Zero self-plays against an executor (Zhao et al., 2025). All of these require the task corpus upfront and multiple sweeps over it (several restart from the base model each round to control drift). Unlike them, Online SDFT improves during a single pass: each problem is seen once, and the model answering problem t+1 is permanently better than the one that answered problem t.

Streaming adaptation without weight updates. The dominant approach to deployment-time improvement stores what was learned outside the model: reflection buffers (Shinn et al., 2023), skill libraries (Wang et al., 2024b), experiential insights (Zhao et al., 2024), induced workflows (Wang et al., 2025), feedback memories (Madaan et al., 2022), and evolving context playbooks; the last of these is framed explicitly as an alternative to parameter updates (Zhang et al., 2025). StreamBench (Wu et al., 2024) canonized streaming evaluation and found Self-StreamICL—a reward-filtered memory retrieved as demonstrations—the

strongest such baseline; it is our ICL baseline and the source of our memory's correctness filter. These methods are capped by the frozen model, lose everything without their memory, and can be misled by retrieved neighbors (§4.2). Unlike them, Online SDFT uses the same filtered memory as a source of supervision, so gains persist in the weights—our mastery evaluation measures exactly this residue after all scaffolding is removed.

Test-time and continual weight adaptation. Test-time training updates weights per instance and discards them (Sun et al., 2020), including LoRA-based variants for few-shot reasoning (Akyürek et al., 2025) and active test-time fine-tuning on retrieved corpus data (Hübotter et al., 2025). Continual-learning methods retain updates but assume task-segmented, labeled datasets (Shi et al., 2025; Parisi et al., 2019), and naive continual fine-tuning forgets (Luo et al., 2023). Unlike transient TTT, our updates are cumulative; unlike CL, our supervision is self-generated from a binary bit with no task boundaries; and the distribution-gap account of forgetting (Yang et al., 2024) explains why our self-generated targets update safely where gold-target fine-tuning would not.

Distillation and RL from verifiable rewards. On-policy distillation trains the student on its own samples against a teacher's distribution (Agarwal et al., 2024; Gu et al., 2024; Xu et al., 2025; Lu, 2025) , but the teacher is an external, stronger, fixed model and training is offline. SDFT rewrites gold datasets into the model's own distribution before fine-tuning (Yang et al., 2024), offline and gold-dependent. SDPO, closest to our objective, makes the feedback-conditioned model a self-teacher inside policy optimization (Hübotter et al., 2026), but remains rollout-batch RL on a fixed prompt distribution with a moving teacher. On the RL side, GRPO, RLOO, and REINFORCE++ optimize the scalar verifiable bit (Shao et al., 2024; Ahmadian et al., 2024; Hu et al., 2025; Ouyang et al., 2022), and entropy/policy collapse on long horizons is a documented failure mode requiring explicit KL control or reference resets (Cui et al., 2025; Liu et al., 2025). Unlike SDPO we distill against a frozen-base teacher—an always-on reference anchor rather than periodic resets—and unlike all of the above we do it online: to our knowledge no prior work performs verified, weight-updating self-distillation over a deployment stream (the nearest streaming weight-update systems are SFT-style without a self-teacher). Our experiments make the contrast operational: the same verified bit that collapses REINFORCE++ on a long mixed stream trains Online SDFT stably (§4.3), and the same loss run as batch epochs underperforms the stream (§4.4).

7 Discussion, Limitations, and Conclusion

Discussion. Most current approaches to deployment-time improvement either accumulate experience outside the model or apply policy gradients to the scalar feedback bit. Our results suggest a third option is viable and often preferable: distilling each verified success into the weights as it happens. The streaming schedule turned out to matter more than we initially expected. It supplies training examples at the model's current frontier, and the re-evaluation step converts weight improvements into additional verified data. Batch training on the same data with the same objective recovers most of the source-task accuracy when given an exploration budget, but we do not yet know whether it matches the stream on generalization; that comparison is running.

Limitations. All experiments use one 7B code-centric model with LoRA adapters and a single stream order per task, so variance across seeds is not characterized. Feedback is binary; richer textual feedback (Hübotter et al., 2026) may carry more signal per interaction and is untested in our setting. Our method issues roughly ten times more oracle calls than the baselines. The extra calls maintain memory rather than affect scoring, but they are a real cost wherever verification is expensive. We also have not measured retention of general capabilities outside the streamed tasks. [TODO: Transfer runs to an unseen domain are in progress.]

Conclusion. We described Online SDFT, a method that updates model weights during a single pass over a problem stream using only binary verifiable feedback. Verified answers are stored in a filtered memory, retrieved as demonstrations, and used as hints for a frozen-base

self-teacher whose next-token distribution the model learns to match. On seven StreamBench tasks the method reaches the highest streaming accuracy of all baselines we tested. On a three-domain stream it improves all domains at once and exceeds the aggregate of three per-domain models, while a REINFORCE++ baseline diverges. We are not aware of prior work that performs verified self-distillation online over a deployment stream.

Reproducibility Statement

All benchmarks are public (StreamBench and its constituent datasets; Table 2 lists sources), and all methods use an open-weights model (Qwen2.5-Coder-7B-Instruct) with fixed released stream orders (seed 42). Complete hyperparameters appear in Appendix B, prompt templates in Appendix E, and oracle definitions in §4.1. Our repository contains the training and evaluation code, per-problem logs for every run in every table, the scripts that generate the paper's tables directly from those logs, and the causal-ordering audit of Appendix A.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In International Conference on Learning Representations, 2024.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, pp. 12248–12267, 2024.

Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. In Proceedings of the 42nd International Conference on Machine Learning, 2025.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In Proceedings of the 41st International Conference on Machine Learning, 2024.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, et al. The entropy mechanism of reinforcement learning for reasoning language models. arXiv preprint arXiv:2505.22617, 2025.

Arsene Fansi Tchango, Rishab Goel, Zhi Wen, Julien Martel, and Joumana Ghosn. DDXPlus: A new dataset for automatic medical diagnosis. Advances in Neural Information Processing Systems, 35, 2022.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In International Conference on Learning Representations, 2024.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. arXiv preprint arXiv:2308.08998, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. Nature, 645:633–638, 2025.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations, 2022.

Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing critic-free policy optimization with global advantage normalization. arXiv preprint arXiv:2501.03262, 2025.

Jonas Hübotter, Sascha Bongni, Ido Hakimi, and Andreas Krause. Efficiently learning at test-time: Active fine-tuning of LLMs. In International Conference on Learning Representations, 2025.

Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. arXiv preprint arXiv:2601.20802, 2026.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. arXiv preprint arXiv:2409.12186, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In International Conference on Learning Representations, 2025.

Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. In Proceedings of the 40th International Conference on Machine Learning, 2023.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. Advances in Neural Information Processing Systems, 36, 2023.

Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. ProRL: Prolonged reinforcement learning expands reasoning boundaries in large language models. In Advances in Neural Information Processing Systems, 2025.

Kevin Lu. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi:10.64434/tml.20251026.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. arXiv preprint arXiv:2308.08747, 2023.

Aman Madaan, Niket Tandon, Peter Clark, and Yiming Yang. Memory-assisted prompt editing to improve GPT-3 after deployment. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 2833–2861, 2022.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In Advances in Neural Information Processing Systems, volume 35, 2022.

German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. Neural Networks, 113:54–71, 2019.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey. ACM Computing Surveys, 58(5), 2025.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, volume 36, 2023.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, et al. Beyond human data: Scaling self-training for problem-solving with language models. Transactions on Machine Learning Research, 2024.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In Proceedings of the 37th International Conference on Machine Learning, 2020.

Boshi Wang, Hao Fang, Jason Eisner, Benjamin Van Durme, and Yu Su. LLMs in the imaginarium: Tool learning through simulated trial and error. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 2024a.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. Transactions on Machine Learning Research, 2024b.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In Proceedings of the 42nd International Conference on Machine Learning, 2025.

Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. Streambench: Towards benchmarking continuous improvement of language agents. In Advances in Neural Information Processing Systems, volume 37, 2024.

Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative knowledge distillation: Bridging the teacher-student gap through interleaved sampling. In International Conference on Learning Representations, 2025.

Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, pp. 1028–1043, 2024.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In Proceedings of the 41st International Conference on Machine Learning, 2024.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825, 2023.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In Advances in Neural Information Processing Systems, volume 35, 2022.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. arXiv preprint arXiv:2510.04618, 2025.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, 2024.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. In Advances in Neural Information Processing Systems, 2025.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning. In Advances in Neural Information Processing Systems, 2025.

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In Advances in Neural Information Processing Systems, 2025.

A Leakage Audit

We verify four causal-ordering invariants programmatically over the raw run logs of the fused experiment (audit script released with the code): (A) every problem is scored exactly once (after de-duplicating checkpoint-resume rows); (B) no memory entry predates its problem's scoring batch—the scored generation can never see its own hint; (C) memory is append-only with respect to the stream, so retrieved demonstrations always come from strictly earlier batches; (D) aggregate counters match the per-problem log exactly. All checks pass on the actual run artifacts. Gold labels exist only inside the 0/1 oracle; the stored hint is always the model's own earlier oracle-passing output, never a dataset field.

Example trace. For a streamed problem, the student prompt contains the system prompt, three retrieved (question, verified-answer) pairs from earlier problems, and the new question—nothing else. The teacher prompt is the identical message list plus one assistant turn containing the model's own stored answer and the question repeated. Full reconstructed traces (built by the training code itself from run artifacts) are released alongside the code.

B Hyperparameters

Base model	Qwen2.5-Coder-7B-Instruct (bf16)
Adapter	LoRA r=16, α=32, dropout 0, all linear layers
Optimizer	AdamW, lr 5×10^{-5}, constant schedule, grad-norm clip 1.0
Distillation	forward KL (α=0), β=0, skip first 3 tokens, temp 1.0
Inner epochs / chunk	2 / grad-accum $\leq$ 50 pairs
Stream	batch 10, window W=9, seed 42
Retrieval	Qwen3-Embedding-0.6B, cosine, k=3, correct-only memory
Decoding	greedy, max 512–1024 new tokens, max sequence 4096
Teacher	frozen base model (never synced)
REINFORCE++	lr 5×10^{-5}, β_{KL}=0.01, PPO clip, whitened adv.
Hardware	1× NVIDIA H200 per run

Table 6: Hyperparameters shared across all experiments.

C Cost Accounting

Per problem, our method spends 1 scored oracle call plus ∼8.8 re-evaluation calls (window re-answers whose only effect is memory update); measured on the fused run: 24,450 total calls for 2,490 problems at the point of measurement (∼9.8/problem). Base, ICL, and REINFORCE++ use exactly 1 call per problem; batch SDFT uses 1 per problem per epoch.

Wall-clock on one H200: standalone streams ∼8–14 h; the fused 4,219-problem run ∼30 h of compute (checkpoint-resumable in preemptible 4 h slices); batch SDFT (4 epochs) ∼3.5 h; a bare-prompt evaluation pass ∼40 min. [TODO: Finalize GPU-hour table per run.]

D Additional Ablation Detail

Generation source. DS-1000 standalone, full 955: bare-prompt generation (offline-SDFT default) .436 acc / 539 regret vs hint-conditioned .423 / 551; the gap accumulates in the last third of the stream. Fused stream (stopped at 1,820 of 4,219 by design): bare-prompt loses ∼10 regret in the first 300 problems, then tracks even for 1,500 problems.

Epochs and learning rate. DS-1000, matched settings: 4 epochs −21 cumulative regret vs 2 epochs; 2 epochs at doubled lr +5 regret vs 2 epochs—epochs do what lr cannot.

Multi-sample distillation. N=10 teacher-conditioned samples per problem, similarity-filtered (sim $\geq$ 0.5 to hint) or unfiltered: both statistically indistinguishable from N=1 on DS-1000 (filtered: 186 vs 188 regret at 300 problems; unfiltered: mildly negative over 520 problems).

REINFORCE++ collapse diagnostics. Batch-level metrics around the collapse (fused stream, batch ∼205–210): approximate KL 0.22 → −0.63, PPO clip fraction 0.08 → 0.68, advantage std 0.38 → 1.37 within 10 batches; running accuracy .503 → .253 by stream end. [TODO: Add full trajectory figures for the collapse diagnostics.]

E Per-Task Prompts

[TODO: Verbatim system prompts and generation templates for all seven tasks, and the student/teacher message structure (available in the released repository; port into figures here).]